Module Interface Specification for TeLaC

Team 24, Maca Boca
Javayria Mudassar
Yilin Jiang
Nour Fawaz
Catherine Yang

November 11, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

See SRS Documentation at https://github.com/Nour-Fawaz/MacaBoca/blob/main/pdfs/SRS/SRS.pdf

[Also add any additional symbols, abbreviations or acronyms —SS]

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Introduction **1**

4 Notation **1**

5 Module Decomposition **1**

6 MIS of [Module Name —SS] **3**
6.1 Module . . . 3
6.2 Uses . . . 3
6.3 Syntax . . . 3
6.3.1 Exported Constants . . . 3
6.3.2 Exported Access Programs . . . 3
6.4 Semantics . . . 3
6.4.1 State Variables . . . 3
6.4.2 Environment Variables . . . 3
6.4.3 Assumptions . . . 3
6.4.4 Access Routine Semantics . . . 3
6.4.5 Local Functions . . . 4

7 MIS of ML Transcription Model **4**
7.1 Module . . . 4
7.2 Uses . . . 4
7.3 Syntax . . . 4
7.3.1 Exported Constants . . . 4
7.3.2 Exported Access Programs . . . 5
7.4 Semantics . . . 5
7.4.1 State Variables . . . 5
7.4.2 Environment Variables . . . 5
7.4.3 Assumptions . . . 5
7.4.4 Access Routine Semantics . . . 6
7.4.5 Local Functions . . . 6

8 MIS of Video Recording Module **7**
8.1 Module . . . 7
8.2 Uses . . . 7
8.3 Syntax . . . 7
8.3.1 Exported Constants . . . 7
8.3.2 Exported Access Programs . . . 7

8.4 Semantics . . . 8
8.4.1 State Variables . . . 8
8.4.2 Environment Variables . . . 8
8.4.3 Assumptions . . . 9
8.4.4 Access Routine Semantics . . . 9
8.4.5 Local Functions . . . 10

9 MIS of Transcription Service Interface Module **11**
9.1 Module . . . 11
9.2 Uses . . . 11
9.3 Syntax . . . 11
9.3.1 Exported Constants . . . 11
9.3.2 Exported Access Programs . . . 11
9.4 Semantics . . . 13
9.4.1 State Variables . . . 13
9.4.2 Environment Variables . . . 13
9.4.3 Assumptions . . . 13
9.4.4 Access Routine Semantics . . . 13
9.4.5 Local Functions . . . 15

10 Appendix — Generative AI Use Disclosure **16**
10.1 AI Tools Used . . . 16
10.2 How and Where Used . . . 16
10.3 Verification of AI-Generated Content . . . 16

11 Appendix — Reflection **17**

3 Introduction

The following document details the Module Interface Specifications for [Fill in your project name and description —SS]

Complementary documents include the System Requirement Specifications and Module Guide. The full documentation and implementation can be found at .... [provide the url for your repo —SS]

4 Notation

[You should describe your notation. You can use what is below as a starting point. —SS]

The structure of the MIS for modules comes from **?**, with the addition that template modules have been adapted from **?**. The mathematical notation comes from Chapter 3 of **?**. For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1|c_2 \Rightarrow r_2|...|c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by TeLaC.

Data Type	Notation	Description
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in (-∞, ∞)
natural number	$\mathbb{N}$	a number without a fractional component in [1, ∞)
real	$\mathbb{R}$	any number in (-∞, ∞)

The specification of TeLaC uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In addition, TeLaC uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

Level 1	**Level 2**
Hardware-Hiding	
Behaviour-Hiding	Input Parameters Output Format Output Verification Temperature ODEs Energy Equations Control Module Specification Parameters Module
Software Decision	Sequence Data Structure ODE Solver Plotting

Table 1: Module Hierarchy

6 MIS of [Module Name —SS]

[Use labels for cross-referencing —SS]

[You can reference SRS labels, such as R??. —SS]

[It is also possible to use LaTeXfor hypperlinks to external documents. —SS]

6.1 Module

[Short name for the module —SS]

6.2 Uses

6.3 Syntax

6.3.1 Exported Constants

6.3.2 Exported Access Programs

Name	In	Out	Exceptions
[accessProg —SS]	-	-	-

6.4 Semantics

6.4.1 State Variables

[Not all modules will have state variables. State variables give the module a memory. —SS]

6.4.2 Environment Variables

[This section is not necessary for all modules. Its purpose is to capture when the module has external interaction with the environment, such as for a device driver, screen interface, keyboard, file, etc. —SS]

6.4.3 Assumptions

[Try to minimize assumptions and anticipate programmer errors via exceptions, but for practical purposes assumptions are sometimes appropriate. —SS]

6.4.4 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]

- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

6.4.5 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

7 MIS of ML Transcription Model

The ML Transcription Model module converts an assessment audio file into timed text segments using a locally run Whisper model, flagging uncertain segments for clinician review (SR??, NFR??).

7.1 Module

MLTransModel

7.2 Uses

- Local file system (to read the audio file)
- Locally installed Whisper speech-to-text model

This module is used by the SAFA module, which supplies the audio file and displays the returned transcript. Storage and transmission of audio and transcripts are handled by other modules (Database, Encryption).

7.3 Syntax

7.3.1 Exported Constants

- CHUNK_DURATION: $\mathbb{R}$
  Length (in seconds) of each increment of audio fed to the model. Each increment yields one segment.
- CONFIDENCE_THRESHOLD: $\mathbb{R}$
  Segments with confidence below this value are flagged for review.

- SUPPORTED_AUDIO_FORMATS: set of String
  Audio file formats the module accepts.
- This module also uses the auxiliary constants MIN_AUDIO_LEVEL (−30 dBFS average) and MAX_REC_LEN (30 minutes), defined in the SRS.

7.3.2 Exported Access Programs

Name	In	Out	Exceptions
transcribe	audioPath: String	transcript: sequence of Segment	InvalidAudioError, ModelUnavailableError

The access programs use the following type:

- Segment: a tuple (text: String, start: $\mathbb{R}$, end: $\mathbb{R}$, confidence: $\mathbb{R}$, flagged: $\mathbb{B}$), representing one displayable chunk of transcribed speech.
  - text is the transcribed speech.
  - start and end give the time range of the chunk in the recording.
  - confidence is a score in $[0, 1]$.
  - flagged is true iff the segment needs clinician review.
- The transcript is a sequence of Segment(s), ordered by start time.

7.4 Semantics

7.4.1 State Variables

None. The module retains no audio or transcript between calls.

7.4.2 Environment Variables

- audioFile: the audio file on the local file system that is being transcribed.
- whisperModel: the locally installed speech-to-text model that converts a chunk of audio into text with a confidence score.

7.4.3 Assumptions

- The model is installed locally and available before transcribe is called.
- audioPath refers to an audio file of one complete assessment session, supplied by the SAFA module.
- The model reports a confidence score for each chunk it transcribes.

- The SAFA module presents flagged segments to the clinician, who reviews them against the video playback before the transcript is used in an assessment (HA-MLTM1).
- This module does not transmit audio or transcripts; transcription runs entirely on the local device, and storage and transmission are handled by the Database and Encryption modules (HA-MLTM2, NFR**??**).
- The audio file is no longer than MAX_REC_LEN.

7.4.4 Access Routine Semantics

transcribe(audioPath):

- transition: None
- output: A sequence of Segments.
  - Divide the audio at audioPath into consecutive chunks of CHUNK_DURATION.
  - For each chunk in order, obtain its text and confidence from whisperModel and form a Segment with the chunk's time range. Set flagged to the value of `shouldFlag` for that segment and chunk.
  - The output are these Segment(s) in time order.
- exception:
  - InvalidAudioError if the file does not exist, is unreadable, is empty, or its format is not in SUPPORTED_AUDIO_FORMATS.
  - ModelUnavailableError if whisperModel cannot be loaded or run.

7.4.5 Local Functions

shouldFlag(s: Segment, chunkAudio):

- transition: None
- output: true if $s.confidence <$ CONFIDENCE_THRESHOLD, or the average audio level of chunkAudio is below MIN_AUDIO_LEVEL (the model cannot reliably recognize speech at that level, so text may be invented or speech missed).
- exception: None

8 MIS of Video Recording Module

The Video Recording module captures an assessment session from the patient's camera and microphone, checks recording quality beforehand, and ensures the recording is saved intact and playable (NFR??, NFR??).

8.1 Module

VideoRecording

8.2 Uses

- Web Platform module (camera and microphone access, and temporary storage on the patient's device)
- Video Storage module (permanent storage, retrieval and backup of recordings)

This module is used by the Web Platform module during an assessment. Encryption of stored recordings is handled by the Encryption module.

8.3 Syntax

8.3.1 Exported Constants

- MIN_BRIGHTNESS: $\mathbb{R}$
  Minimum average video brightness for the lighting to be considered adequate.
- MIN_FRAME_RATE: $\mathbb{R}$
  Minimum video frame rate (frames per second) accepted.
- MAX_UPLOAD_ATTEMPTS: $\mathbb{N}$
  Number of times saving is attempted automatically before the failure is reported.
- This module also uses the auxiliary constants MIN_AUDIO_LEVEL (-30 dBFS average), MIN_VIDEO_RESOLUTION (640×480 pixels) and MAX_REC_LEN (30 minutes), defined in the SRS.

8.3.2 Exported Access Programs

Name	In	Out	Exceptions
checkQuality	-	report: QualityReport	PermissionDeniedError, DeviceUnavailableError
startRecording	-	-	PermissionDeniedError, DeviceUnavailableError, RecordingInProgressError
stopRecording	-	-	NotRecordingError
saveRecording	assessmentID: String	-	NoRecordingError, UploadFailedError, VerificationFailedError
getStatus	-	status: {IDLE, RECORDING, PENDING_SAVE, SAVED}	-

The access programs use the following type:

- QualityReport: a tuple (passed: $\mathbb{B}$, issues: sequence of String).
  - passed is true iff the audio and video meet the minimum levels above.
  - issues describes each problem found (e.g., audio too quiet, lighting too dark, resolution too low) so the patient can correct their setup.

8.4 Semantics

8.4.1 State Variables

- status: {IDLE, RECORDING, PENDING_SAVE, SAVED}
  Where the current recording is in its lifecycle.
- localCopy: Recording or none
  The temporary copy of the current recording held on the patient's device until the system confirms it was saved successfully.

8.4.2 Environment Variables

- camera: the patient's video input device.
- microphone: the patient's audio input device.
- deviceStorage: temporary storage on the patient's device, in which localCopy is held.

8.4.3 Assumptions

- The caller runs checkQuality, and the patient has corrected any reported issues, before startRecording is called (HA-VR1).
- The Video Storage module reports success or failure of a store request, can return a stored recording for verification, and can create a backup copy of a stored recording.
- The patient's device has enough temporary storage to hold the full recording.
- assessmentID is supplied by the caller and identifies the assessment the recording belongs to.

8.4.4 Access Routine Semantics

checkQuality():

- transition: None
- output: A QualityReport based on a short sample of the camera and microphone input. passed is true if the sample's audio level is at least MIN_AUDIO_LEVEL, its brightness is at least MIN_BRIGHTNESS, and its resolution and frame rate are at least MIN_RESOLUTION and MIN_FRAME_RATE. Each failed condition adds an entry to issues. The sample is discarded.
- exception: PermissionDeniedError if access to the camera or microphone is not granted; DeviceUnavailableError if either device is missing or in use elsewhere.

startRecording():

- transition: status := RECORDING; begin capturing audio and video from camera and microphone.
- output: None
- exception: PermissionDeniedError if access to the camera or microphone is not granted; DeviceUnavailableError if either device is missing or in use elsewhere; RecordingInProgressError if status = RECORDING.

stopRecording():

- transition: Stop capturing. localCopy := the complete recording, saved in deviceStorage. status := PENDING_SAVE.
- output: None
- exception: NotRecordingError if status $\neq$ RECORDING.

saveRecording(assessmentID):

- transition:
  - Request that Video Storage store localCopy under assessmentID, trying up to MAX_UPLOAD_ATTEMPTS times.
  - Retrieve the stored recording and check it with `isIntact`.
  - If the check passes, request that Video Storage create a backup of the stored recording, then localCopy := none and status := SAVED.
  - If any step fails, localCopy and status are unchanged, so saveRecording can be called again.
- output: None
- exception: NoRecordingError if status $\neq$ PENDING_SAVE; UploadFailedError if storing the recording did not succeed after MAX_UPLOAD_ATTEMPTS attempts; VerificationFailedError if the stored recording is incomplete or cannot be played back.

getStatus():

- transition: None
- output: status
- exception: None

8.4.5 Local Functions

isIntact(stored: Recording, original: Recording):

- transition: None
- output: true if stored can be decoded and played from start to end, and its duration and content match original.
- exception: None

9 MIS of Transcription Service Interface Module

The Transcription Service Interface module generates a draft transcript for an assessment recording and manages the clinician's review of it: edits, speaker roles and verification (FR??, FR??, FR??, NFR??).

9.1 Module

TranscriptionServiceInterface

9.2 Uses

- MLTransModel module (generates timed text segments from audio)
- Video Storage module (provides the recording of an assessment)
- Database module (stores transcripts and records changes to them in the audit log)

This module is used by the Web Platform module, which displays the transcript and passes the clinician's actions to it.

9.3 Syntax

9.3.1 Exported Constants

- SPEAKER_ROLES: set of String = {System, Child, Parent, Experimenter, Multiple Speakers, Others}
  The roles a clinician can assign to a speaker.
- GENERATION_ATTEMPTS: $\mathbb{N}$
  Number of times transcription is attempted automatically before a failure is reported, so the clinician does not have to restart it manually.
- This module also relies on the auxiliary constants MAX_REC_LEN (30 minutes) and SYNC_TOL (500 ms), defined in the SRS.

9.3.2 Exported Access Programs

Name	In	Out	Exceptions
generateTranscript	assessmentID: String, userID: String, portion: Interval or none	-	AssessmentNotFoundError, RecordingUnavailableError, TranscriptionInProgressError, PortionAlreadyTranscribedError, GenerationFailedError
getTranscript	assessmentID: String, userID: String	transcript: Transcript	AssessmentNotFoundError, NoTranscriptError
editSegmentText	assessmentID: String, segmentID: String, newText: String, userID: String	-	NoTranscriptError, SegmentNotFoundError, TranscriptionInProgressError, SaveFailedError
assignSpeaker	assessmentID: String, segmentID: String, role: String, userID: String	-	NoTranscriptError, SegmentNotFoundError, InvalidRoleError, TranscriptionInProgressError, SaveFailedError
verifySegment	assessmentID: String, segmentID: String, userID: String	-	NoTranscriptError, SegmentNotFoundError, TranscriptionInProgressError, SaveFailedError
completeTranscript	assessmentID: String, userID: String	unverified: sequence of String	NoTranscriptError, TranscriptionInProgressError, SaveFailedError

The access programs use the following types:

- Interval: a pair of times (from, to) in seconds on the recording's timeline.
- Transcript: a pair (status, segments).
  - status is one of {BEING_GENERATED, DRAFT, VERIFIED}.
  - segments is a sequence of Segment(s), ordered by start time.
- Segment: a tuple (segmentID: String, start: $\mathbb{R}$, end: $\mathbb{R}$, text: String, originalText: String, speaker: String or none, language: String, confidence: $\mathbb{R}$, flagged: $\mathbb{B}$, verified: $\mathbb{B}$).
  - Times are stored to the nearest tenth of a second or better so playback can be synchronized within SYNC_TOL (NFR??).

  - A segment counts as edited when text differs from originalText, so automatic and clinician-modified content can be told apart (NFR??).

9.4 Semantics

9.4.1 State Variables

None. All transcript data is held by the Database module, so work saved in one session is available in the next (NFR??).

9.4.2 Environment Variables

None.

9.4.3 Assumptions

- The caller has confirmed that userID belongs to the clinician assigned to the patient of the assessment (NFR??).
- Each assessment has one recording, up to MAX_REC_LEN long, retrievable from Video Storage.
- The Database module records an audit entry for every view or change of a transcript made through this module (NFR??).
- Clinicians review flagged segments against the video playback before relying on the transcript (HA-MLTM1).
- Encryption of recordings and transcripts in transit and in storage is provided by the Encryption, Video Storage and Database modules (NFR??).

9.4.4 Access Routine Semantics

generateTranscript(assessmentID, userID, portion):

- transition:
  - Set the transcript status to BEING_GENERATED.
  - Obtain the assessment's recording from Video Storage and extract the audio of the whole recording, or of portion if given, into a temporary audio file (`extractAudio`).
  - Call MLTransModel.transcribe on the audio file, retrying up to GENERATION_ATTEMPTS times if the model is unavailable.
  - Store each returned segment with text and originalText equal to the generated text, speaker none, verified false, and its times shifted by the start of portion.
  - Set the transcript status to DRAFT.

  - Delete the temporary audio file, whether or not generation succeeded.
  - If generation fails, remove any segments added by this call and restore the previous status.

- output: None
- exception: AssessmentNotFoundError if assessmentID does not exist; RecordingUnavailableError if the recording cannot be obtained; TranscriptionInProgressError if the status is already BEING_GENERATED; PortionAlreadyTranscribedError if existing segments overlap portion (or any segments exist, when portion is none); GenerationFailedError if all attempts fail.

getTranscript(assessmentID, userID):

- transition: None
- output: The stored transcript, including its status.
- exception: AssessmentNotFoundError if assessmentID does not exist; NoTranscriptError if no transcript has been generated for it.

editSegmentText(assessmentID, segmentID, newText, userID):

- transition: Set text of the segment to newText. originalText is unchanged. The change is saved immediately.
- output: None
- exception: NoTranscriptError, SegmentNotFoundError, TranscriptionInProgressError if the status is BEING_GENERATED; SaveFailedError if the change could not be saved, in which case nothing is changed and the caller keeps the clinician's input to retry.

assignSpeaker(assessmentID, segmentID, role, userID):

- transition: Set speaker of the segment to role. The change is saved immediately.
- output: None
- exception: InvalidRoleError if role is not in SPEAKER_ROLES, otherwise same as editSegmentText.

verifySegment(assessmentID, segmentID, userID):

- transition: Set verified of the segment to true. The change is saved immediately.
- output: None
- exception: Same as editSegmentText.

completeTranscript(assessmentID, userID):

- transition: Set the transcript status to VERIFIED. Segments that are not verified keep verified false.
- output: The segmentIDs of the segments that are still unverified, so the caller can offer to return to them.
- exception: NoTranscriptError, TranscriptionInProgressError, SaveFailedError.

9.4.5 Local Functions

extractAudio(recording, portion):

- transition: Writes a temporary audio file.
- output: A file in a format supported by MLTransModel containing the audio of recording (restricted to portion, if given).
- exception: RecordingUnavailableError if the recording has no readable audio.

10 Appendix — Generative AI Use Disclosure

This section documents all the uses of generative artificial intelligence used throughout the document.

10.1 AI Tools Used

- Claude Sonnet 5 (Medium)
- ChatGPT-5.6 Luna

10.2 How and Where Used

The following lists the uses of AI applied throughout the entire document:

- Fixing grammar and spelling
- Debugging possible LaTeX errors
- Performing a final check over the document at the end, comparing each section against the original document comments and the rubric to ensure nothing was missed
- Removing template comments

Section 5

- Format the table of existing modules for LaTeX

All Modules

- Brainstorm modules in addition to the already existing ones
- Flesh out ideas for each module

10.3 Verification of AI-Generated Content

To validate the LLM's output, each team member reviewed their assigned section and presented it to the group, summarizing the work they had completed since the last meeting. All members were able to explain their sections clearly. Before any pull request was approved, a team member who had not authored the section reviewed it to confirm that it was cohesive, accurate, and consistent with the rest of the document. Because the document contained no factual claims, the LLM's output did not require any additional fact-checking.

11 Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?

4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?

5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)

6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

7. (After you have implemented another team's module, which means this isn't filled in until after the original deadline). What did you learn by implementing another team's module? Were all the details you needed in the documentation, or did you need to make assumptions, or ask the other team questions? If your team also had another team implement one of your modules, what was this experience like? Are there things in your documentation you could have changed to make the process go more smoothly for when an "outsider" completes some of the implementation?

Your reflection should be about concepts learned in your SE education, such as modular decomposition, cohesion, coupling, information hiding, etc. You should critique the quality and utility of the specification you received.

You should create a PR for your implemented module. A link to the PR should be part of your reflection.

Nour

Question 1

Writing the MIS went smoothly because we already had a clear understanding of what each module is meant to do and how it contributes to the system. Our requirements and hazard analysis gave us a solid starting point, so we could focus on defining each module's interface instead of deciding what the module was for. Knowing each module's intended behaviour made it easier to decide what to expose through its access programs and what to keep hidden.

Question 2

The most difficult part was figuring out how the different modules would interact considering several of them depend on each other. For example, the Transcription Service Interface relies on Video Storage, the ML Transcription Model and the Database. It was hard to decide which module should own each responsibility and where one module's interface should end and the next begin. We resolved this by discussing the data that passes between modules and keeping each interface to the minimum its callers need. This kept coupling low and let us write each module without knowing how the others work internally.

Question 3

The ML Transcription Model decision came from our supervisors. They already use Whisper in their existing workflow, but they access it through an API. However, they required us to implement the system such that the model is run locally. This changes the module's interface and means recordings do not have to leave the device for transcription.

The decomposition of the remaining modules were our own decisions. We split them so that each module has a single responsibility, without breaking the system down so far that the modules became trivial. I worked on the Video Recording and Transcription Service Interface modules. We made each its own module because they handle different concerns. Video Recording deals with capturing and verifying recordings, and the Transcription Service Interface manages generating and reviewing transcripts.

Question 4

None of our other documents really needed to be changed. In fact, we referenced our requirements and hazard analysis while deciding on the different modules of the system, hence they were useful as is. Our supervisors also checked over our requirements and were satisfied with them. That told us we were on the right track, so we went ahead and started designing our system.

Question 5

The biggest limitation is scope. Some features in our requirements document, such as result visualization and patient-facing reports, are listed under likely changes. They are low priority, so we don't expect to build them within this project and have documented them as future work.

The other limitation is the transcription model itself. We use Whisper, and its performance is average, particularly for difficult Mandarin words. We reduced the risk by flagging low-confidence segments and having the clinician review and correct the transcript. It still means the transcripts won't always be accurate on the first pass.

With unlimited time and resources, we would first add an interference detection system using a machine learning model. We would also evaluate how accurate the transcription is on Mandarin-English speech, for example by comparing the generated transcripts with the clinicians' corrected versions.

Question 6

The first alternative design we thought of was combining recording, transcription and storage into one large module. The benefit would be fewer interfaces to manage, and there would be no need to decide how modules communicate. However, the trade-off is that the module would have low cohesion, so a change in one area (for example, transcript editing, which we expect to change) could affect the others. We didn't choose it because separate modules let each one hide its own design decisions and be tested on its own.

The second alternative design was having the transcription module keep the transcript in memory. This would be simpler to write; however, edits could be lost between sessions, and clinicians might not be able to continue their work from another device. We didn't choose it because our transcription module stores nothing itself and saves all transcript data through the Database module, so the saved work is preserved.

Overall, we chose our design because it separates responsibilities cleanly, so each module can be changed and tested on its own. This also allows each module to be small enough to build in the time we have.